



Chapter 15

Project 5.1: Modeling Base Application

The next step in the pipeline from acquisition to clean-and-convert is the analysis and some preliminary modeling of the data. This may lead us to use the data for a more complex model or perhaps machine learning.

This chapter will guide you through creating another application in the three-stage pipeline to acquire, clean, and model a collection of data. This first project will create the application with placeholders for more detailed and application-specific modeling components. This makes it easier to insert small statistical models that can be replaced with more elaborate processing if needed.

In this chapter, we'll look at two parts of data analysis:

- CLI architecture and how to design a more complex pipeline of processes for gathering and analyzing data
- The core concepts of creating a statistical model of the data

Viewed from a distance, all analytical work can be considered to be creating a simplified model of important features of some complicated processes. Even something as simple-sounding as computing an average suggests a simple model of the central tendency of a variable. Adding a standard deviation suggests an expected range for the variable's values and – further – assigns a probability to values outside the range.

Models, can, of course, be considerably more detailed. Our purpose is to start down the path of modeling in a way that builds flexible, extensible application software. Each application will have unique modeling re-

quirements, depending on the nature of the data, and the nature of the questions being asked about the data. For some processes, means and standard deviations are adequate for spotting outliers. For other processes, a richer and more detailed simulation may be required to estimate the expected distribution of data.

We'll start the modeling by looking at variables in isolation, sometimes called *univariate* statistics. This will examine a variety of commonly recognized distributions of data. These distributions generally have a few parameters that can be discovered from the given data. In this chapter, we'll also look at measures like mean, median, standard deviation, variance, and standard deviation. These can be used to describe data that has a normal or Gaussian distribution. The objective is to create a CLI application separate from an analytic notebook used to present results. This creates a higher degree of automation for modeling. The results may then be presented in an analytical notebook.

There is a longer-term aspect to having automated model creation. Once a data model has been created, an analyst can also look at changes to a model and what implications the changes should have on the way an enterprise operates. For example, an application may perform a monthly test to be sure new data matches the established mean, median, and standard deviation reflecting the expected normal distribution of data. In the event that a batch of data doesn't fit the established model, further investigation is required to uncover the root cause of this change.

15.1 Description

This application will create a report on a dataset presenting a number of statistics. This automates the ongoing monitoring aspect of an Analysis Notebook, reducing the manual steps and creating reproducible results. The automated computations stem from having a statistical model for the data, often created in an analysis notebook, where alternative models are explored. This reflects variables with values in an expected range.

For industrial monitoring, this is part of an activity called **Gage repeatability and reproducibility**. The activity seeks to confirm that measure-

ments are repeatable and reproducible. This is described as looking at a “measurement instrument.” While we often think of an instrument as being a machine or a device, the definition is actually very broad. A survey or questionnaire is a measurement instrument focused on people’s responses to questions.

When these computed statistics deviate from expectations, it suggests something has changed, and the analyst can use these unexpected values to investigate the root cause of the deviation. Perhaps some enterprise process has changed, leading to shifts in some metrics. Or, perhaps some enterprise software has been upgraded, leading to changes to the source data or encodings used to create the clean data. More complex still, it may be that the instrument doesn’t actually measure what we thought it measured; this new discrepancy may expose a gap in our understanding.

The repeatability of the model’s measurements is central to the usability of the measurements. Consider a ruler that’s so worn down over years of use that it is no longer square or accurate. This single instrument will produce different results depending on what part of the worn end is used to make the measurement. This kind of measurement variability may obscure the variability in manufacturing a part. Understanding the causes of changes is challenging and can require thinking “outside the box” — challenging assumptions about the real-world process, the measurements of the process, and the model of those measurements.

Exploratory data analysis can be challenging and exhilarating precisely because there aren’t obvious, simple answers to explain why a measurement has changed.

The implementation of this preliminary model is through an application, separate from the previous stages in the pipeline to acquire and clean the data. With some careful design, this stage can be combined with those previous stages, creating a combined sequence of operations to acquire, clean, and create the summary statistics.

This application will overlap with the analysis notebook and the initial inspection notebook. Some of the observations made during those earlier

ad-hoc analysis stages will be turned into fixed, automated processing.

This is the beginning of creating a more complicated machine-learning model of the data. In some cases, a statistical model using linear or logistic regression is adequate, and a more complex artificial intelligence model isn't needed. In other cases, the inability to create a simple statistical model can point toward a need to create and tune the hyperparameters of a more complicated model.

The objective of this application is to save a statistical summary report that can be aggregated with and compared to other summary reports. The ideal structure will be a document in an easy-to-parse notation. JSON is suggested, but other easier-to-read formats like TOML are also sensible.

There are three key questions about data distribution:

1. What is the **location** or expected value for the output being measured?
2. What is the **spread** or expected variation for this variable?
3. What is the general **shape**, e.g., is it symmetric or skewed in some way?

For more background on these questions, see

<https://www.itl.nist.gov/div898/handbook/ppc/section1/ppc131.htm>

This summary processing will become part of an automated acquire, clean, and summarize operation. The **User Experience (UX)** will be a command-line application. Our expected command line should look something like the following:

```
% python src/summarize.py -o summary/series_1/2023/03 data/clean/Series_1.ndj
```

The `-o` option specifies the path to an output sub-directory. The output filename added to this path will be derived from the source file name. The source file name often encodes information on the applicable date range for the extracted data.

The Anscombe's Quartet data doesn't change and wouldn't really have an "applicable date" value.

We've introduced the **idea** of periodic enterprise extractions. None of the projects actually specify a data source subject to periodic change.

Some web services like <http://www.yelp.com> have health-code data for food-service businesses; this is subject to periodic change and serves as a good source of analytic data.

Now that we've seen the expectations, we can turn to an approach to the implementation.

15.2 Approach

We'll take some guidance from the C4 model (<https://c4model.com>) when looking at our approach:

- **Context:** For this project, a context diagram would show a user creating analytical reports. You may find it helpful to draw this diagram.
- **Containers:** There only seems to be one container: the user's personal computer.
- **Components:** We'll address the components below.
- **Code:** We'll touch on this to provide some suggested directions.

The heart of this application is a module to summarize data in a way that lets us test whether it fits the expectations of a model. The statistical model is a simplified reflection of the underlying real-world processes that created the source data. The model's simplifications include assumptions about events, measurements, internal state changes, and other details of the processing being observed.

For very simple cases — like Anscombe's Quartet data — there are only two variables, which leaves a single relationship in the model. Each of the four sample collections in the quartet has a distinct relationship. Many of the summary statistics, however, are the same, making the relationship often surprising.

For other datasets, with more variables and more relationships, there are numerous choices available to the analyst. The *NIST Engineering Statistics Handbook* has an approach to modeling. See <https://www.itl.nist.gov/div898/handbook/index.htm> for the design of a model and analysis of the results of the model.

As part of the preliminary work, we will distinguish between two very broad categories of statistical summaries:

- **Univariate statistics:** These are variables viewed in isolation.
- **Multivariate statistics:** These are variables in pairs (or higher-order groupings) with an emphasis on the relationship between the variable's values.

For univariate statistics, we need to understand the distribution of the data. This means measuring the location (the center or expected values), the spread (or scale), and the shape of the distribution. Each of these measurement areas has several well-known statistical functions that can be part of the summary application.

We'll look at the multivariate statistics in the next chapter. We'll start the univariate processing by looking at the application in a general way, and then focus on the statistical measures, the inputs, and finally, the outputs.

15.2.1 Designing a summary app

This application has a command-line interface to create a summary from the cleaned data. The input file(s) are the samples to be summarized. The summary must be in a form that's easy to process by subsequent software. This can be a JSON- or a TOML-formatted file with the summary data.

The summaries will be "measures of location," sometimes called a "central tendency." See <https://www.itl.nist.gov/div898/handbook/eda/section3/eda351.htm>.

The output must include enough context to understand the data source, and the variable being measured. The output also includes the measured values to a sensible number of decimal places. It's important to avoid introducing additional digits into floating-point values when those digits are little more than noise.

A secondary feature of this application is to create an easy-to-read presentation of the summary. This can be done by using tools like **Docutils** to transform a reStructuredText report into HTML or a PDF. A tool like **Pandoc** could also be used to convert a source report into something that isn't simply text. The technique explored in [*Chapter 14, Project 4.2: Creating Reports*](#) is to use Jupyter{Book} to create a document suitable for publication.

We'll start by looking at some of the measures of location that need to be computed.

15.2.2 Describing the distribution

As noted above, there are three aspects of the distribution of a variable. The data tends to scatter around a central tendency value; we'll call this the location. There will be an expected limit on the scattering; we'll call this the spread. There may be a shape that's symmetric or skewed in some way. The reasons for scattering may include measurement variability, as well as variability in the process being measured.

The NIST Handbook defines three commonly-used measures of location:

- **mean:** The sum of the variable's values divided by the count of values:
$$X = \frac{\sum X_i}{N}.$$
- **median:** The value of a value that is in the center of the distribution. Half the values are less than or equal to this value, and half the values are greater than or equal to this value. First, sort the values into ascending order. If there's an odd number, it's the value in the center. For an even number of values, split the difference between the two center-most values.

- **mode:** The most common value. For some of the Anscombe Quartet data series, this isn't informative because all of the values are unique.

These functions are first-class parts of the built-in `statistics` module, making them relatively easy to compute.

There are some alternatives that may be needed when the data is polluted by outliers. There are techniques like *Mid-Mean* and *Trimmed Mean* to discard data outside some range of percentiles.

The question of an "outlier" is a sensitive topic. An outlier may reflect a measurement problem. An outlier may also hint that the processing being measured is quite a bit more complicated than is revealed in a set of samples. Another, separate set of samples may reveal a different mean or a larger standard deviation. The presence of outliers may suggest more study is needed to understand the nature of these values.

There are three commonly-used measures for the scale or spread of the data:

- **Variance** and standard deviation. The variance is — essentially — the average of the squared distance of each sample from the mean: $s^2 = \frac{\sum (X_i - \bar{X})^2}{(N-1)}$. The standard deviation is the square root of the variance.
- **Range** is the difference between the largest and smallest values.
- **Median absolute deviation** is the median of the distance of each sample from the mean: $MAD_Y = \text{median}(|Y_i - \bar{Y}|)$. See [*Chapter 7, Data Inspection Features*](#).

The variance and standard deviation functions are first-class parts of the built-in `statistics` module. The range can be computed using the built-in `min()` and `max()` functions. A median absolute deviation function can be built using functions in the `statistics` module.

There are also measures for skewness and kurtosis of a distribution. We'll leave these as extras to add to the application once the base statistical measures are in place.

15.2.3 Use cleaned data model

It's essential to use the cleaned, normalized data for this summary processing. There is some overlap between an inspection notebook and this more detailed analysis. An initial inspection may also look at some measures of location and range to determine if the data can be used or contains errors or problems. During the inspection activities, it's common to start creating an intuitive model of the data. This leads to formulating hypotheses about the data and considering experiments to confirm or reject those hypotheses.

This application formalizes hypothesis testing. Some functions from an initial data inspection notebook may be refactored into a form where those functions can be used on the cleaned data. The essential algorithm may be similar to the raw data version of the function. The data being used, however, will be the cleaned data.

This leads to a sidebar design decision. When we look back at the data inspection notebook, we'll see some overlaps.

15.2.4 Rethink the data inspection functions

Because Python programming can be generic — independent of any specific data type — it's tempting to try to unify the raw data processing and the cleaned data processing. The desire manifests as an attempt to write exactly one version of some algorithm, like the Median Absolute Deviation function that's usable for *both* raw and cleaned data.

This is not always an achievable goal. In some situations, it may not even be desirable.

A function to process raw data must often do some needed cleaning and filtering. These overheads are later refactored and implemented in the pipeline to create cleaned data. To be very specific, the `if` conditions used to exclude bad data can be helpful during the inspection. These conditions will become part of the clean-and-convert applications. Once this is done, they are no longer relevant for working with the cleaned data.

Because the extra data cleanups are required for inspecting raw data, but not required for analyzing cleaned data, it can be difficult to create a single process that covers both cases. The complications required to implement this don't seem to be worth the effort.

There are some additional considerations. One of these is the general design pattern followed by Python's `statistics` module. This module works with sequences of atomic values. Our applications will read (and write) complicated `Sample` objects that are not atomic Python integer or float values. This means our applications will extract sequences of atomic values from sequences of complicated `Sample` objects.

The raw data, on the other hand, may not have a very sophisticated class definition. This means the decomposition of complicated objects isn't part of the raw data processing.

For some very, very large datasets the decomposition of complicated multivariate objects to individual values may happen as the data is being read. Rather than ingest millions of objects, the application may extract a single attribute for processing.

This might lead to input processing that has the following pattern:

```
from collections.abc import Iterator, Callable
import json
from pathlib import Path
from typing import TypeAlias

from analysis_model import Sample

Extractor: TypeAlias = Callable[[Sample], float]

def attr_iter(some_path: Path, extractor: Extractor) -> Iterator[float]:
    with some_path.open() as source:
        for line in source:
            document = Sample(**json.loads(line))
            yield extractor(document)
```

```
def x_values(some_path: Path) -> list[float]:  
    return list(attr_iter(some_path, lambda sample: sample.x))
```

This example defines a generic function, `attr_iter()`, to read an NDJSON file to build instances of some class, `Sample`. (The details of the `Sample` class are omitted.)

The `x_values()` function uses the generic `attr_iter()` function with a concrete lambda object to extract a specific variable's value, and create a list object. This list object can then be used with various statistical functions.

While a number of individual `Sample` objects are created, they aren't retained. Only the values of the `x` attribute are saved, reducing the amount of memory used to create summary statistics from a large collection of complicated values.

15.2.5 Create new results model

The statistical summary contains three broad kinds of data:

- Metadata to specify what source data is used to create the summary.
- Metadata to specify what measures are being used.
- The computed values for location, shape, and spread.

In some enterprise applications, source data is described by a range of dates defining the earliest and latest samples. In some cases, more details are required to describe the complete context. For example, the software to acquire raw data may have been upgraded in the past. This means older data may be incomplete. This means the context for processing data may require some additional details on software versions or releases in addition to the range of dates and data sources.

Similarly, the measures being used may shift over time. The computation of skewness, for example, may switch from the **Fisher-Pearson** formula to the **adjusted Fisher-Pearson** formula. This suggests the version information for the summary program should also be recorded along with the results computed.

Each of these metadata values provides necessary context and background information on the data source, the method of collection, and any computations of derived data. This context may be helpful in uncovering the root cause of changes. In some cases, the context is a way to catalog underlying assumptions about a process or a measurement instrument; seeing this context may allow an analyst to challenge assumptions and locate the root cause of a problem.

The application must create a result document that looks something like the following example:

```
[identification]
  date = "2023-03-27T10:04:00"
[creator]
  title = "Some Summary App"
  version = 4.2
[source]
  title = "Anscombe's Quartet"
  path = "data/clean/Series_1.ndj"
[x.location]
  mean = 9.0
[x.spread]
  variance = 11.0
[y.location]
  mean = 7.5
[y.spread]
  variance = 4.125
```

This file can be parsed by the `toml` or `tomllib` module to create a nested collection of dictionaries. The secondary feature of the summary application is to read this file and write a report, perhaps using Markdown or ReStructuredText that provides the data in a readable format suitable for publication.

For Python 3.11 or newer, the `tomllib` module is built in. For older Python installations, the `toml` module needs to be installed.

Now that we've seen the overall approach, we can look at the specific deliverable files.

15.3 Deliverables

This project has the following deliverables:

- Documentation in the `docs` folder.
- Acceptance tests in the `tests/features` and `tests/steps` folders.
- Unit tests for model module classes in the `tests` folder.
- Mock objects for the `csv_extract` module tests will be part of the unit tests.
- Unit tests for the `csv_extract` module components that are in the `tests` folder.
- An application to summarize the cleaned data in a TOML file.
- An application secondary feature to transform the TOML file to an HTML page or PDF file with the summary.

In some cases, especially for particularly complicated applications, the summary statistics may be best implemented as a separate module. This module can then be expanded and modified without making significant changes to the overall application.

The idea is to distinguish between these aspects of this application:

- The CLI, which includes argument parsing and sensible handling of input and output paths.
- The statistical model, which evolves as our understanding of the problem domain and the data evolve.
- The data classes, which describe the structure of the samples, independent of any specific purpose.

For some applications, these aspects do not involve a large number of classes or functions. In a case where the definitions are small, a single Python module will do nicely. For other applications, particularly those where initial assumptions turned out to be invalid and significant changes were made, having separate modules can permit more flexibility, and more agility with respect to future changes.

We'll look at a few of these deliverables in a little more detail. We'll start with some suggestions for creating the acceptance tests.

15.3.1 Acceptance testing

The acceptance tests need to describe the overall application's behavior from the user's point of view. The scenarios will follow the UX concept of a command-line application to acquire data and write output files.

Because the input data has been cleaned and converted, there are few failure modes for this application; extensive testing of potential problems isn't as important as it was in earlier data-cleaning projects.

For relatively simple datasets, the results of the statistical summaries are known in advance. This leads to features that might look like the following example:

Feature: Summarize an Anscombe Quartet Series.

Scenario: When requested, the application creates a TOML summary of a series.

Given the "clean/series_1.ndj" file exists

When we run command "python src/summarize.py \

-o summary/series_1/2023/03 data/clean/Series_\1.ndj"

Then the "summary/series_1/2023/03/summary.toml" file exists

And the value of "summary['creator']['title']" is "Anscombe Summary App"

And the value of "summary['source']['path']" is "data/clean/Series_1.ndj"

And the value of "summary['x']['location']['mean']" is "9.0"

We could continue the scenario with a number of additional **Then** steps to validate each of the locations and the spread and shape the statistical summaries.

The step definitions will be similar to step definitions for a number of previous projects. Specifically, the **When** step will use the `subprocess.run()` function to execute the given application with the required command-line arguments.

The first of the **Then** steps will need to read — and parse — the TOML file. The resulting summary object can be placed in the **context** object.

Subsequent **Then** steps can examine the structure to locate the individual values, and confirm the values match the acceptance test expectations.

It is often helpful to extract a small subset of data to use for acceptance testing. Instead of processing millions of rows, a few dozen rows are adequate to confirm the application has read and summarized data. The data only needs to be representative of the larger set of samples under consideration.

Because the chosen subset is part of the testing suite; it rarely changes. This makes the results predictable.

As the data collection process evolves, it's common to have changes to the data sources. This will lead to changes in the data cleaning. This may, in turn, lead to changes in the summary application as new codes or new outliers must be handled properly. The evolution of the data sources implies that the test data suite will also need to evolve to expose any of the special, edge, or corner cases.

Ideally, the test data suite is a mixture of ordinary — no surprises — data, mixed with representative examples of each of the special, atypical cases. As this test data suite evolves, the acceptance test scenario will also evolve.

The TOML file is relatively easy to parse and verify. The secondary feature of this application — expanding on the TOML output to add extensive Markdown — also works with text files. This makes it relatively easy to confirm with test scenarios that read and write text files.

The final publication, whether done by Pandoc or a combination of Pandoc and a LaTeX toolchain, isn't the best subject for automated testing. A good copy editor or trusted associate needs to make sure the final document meets the stakeholder's expectations.

15.3.2 Unit testing

It's important to have unit testing for the various components that are unique to this application. The clean data class definition, for example, is created by another application, with its own test suite. The unit tests for this application don't need to repeat those tests. Similarly, the `statistics` module has extensive unit tests; this application's unit tests do not need to replicate any of that testing.

This further suggests that the `statistics` module should be replaced with `Mock` objects. Those mock objects can — generally — return `sentinel` objects that will appear in the resulting TOML-format summary document.

This suggests test cases structured like the following example:

```
from pytest import fixture
from unittest.mock import Mock, call, sentinel
import summary_app

@fixture
def mocked_mean(monkeypatch):
    mean = Mock(
        return_value=sentinel.MEAN
    )
    monkeypatch.setattr(summary_app, 'mean', mean)
    return mean

@fixture
def mocked_variance(monkeypatch):
    variance = Mock(
        return_value=sentinel.VARIANCE
    )
    monkeypatch.setattr(summary_app, 'variance', variance)
    return variance

def test_var_summary(mocked_mean, mocked_variance):
    sample_data = sentinel.SAMPLE
    result = summary_app.variable_summary(sample_data)
    assert result == {
        "location": {"mean": sentinel.MEAN},
        "spread": {"variance": sentinel.VARIANCE},
```

```
}  
assert mocked_mean.mock_calls == [call(sample_data)]  
assert mocked_variance.mock_calls == [call(sample_data)]
```

The two test fixtures provide mock results, using `sentinel` objects. Using `sentinel` objects allows easy comparison to be sure the results of the mocked functions were not manipulated unexpectedly by the application.

The test case, `test_var_summary()`, provides a mocked source of data in the form of another `sentinel` object. The results have the expected structure and the expected `sentinel` objects.

The final part of the test confirms the sample data — untouched — was provided to the mocked statistical functions. This confirms the application doesn't filter or transform the data in any way. The results are the expected `sentinel` objects; this confirms the module didn't adulterate the results of the `statistics` module. And the final check confirms that the mocked functions were called exactly once with the expected parameters.

This kind of unit test, with numerous mocks, is essential for focusing the testing on the new application code, and avoiding tests of other modules or packages.

15.3.3 Application secondary feature

A secondary feature of this application transforms the TOML summary into a more readable HTML or PDF file. This feature is a variation of the kinds of reporting done with Jupyter Lab (and associated tools like **Jupyter {Book}**).

There's an important distinction between these two classes of reports:

- The Jupyter Lab reports involve discovery. The report content is always new.
- The summary application's reports involve confirmation of expectations. The report content should not be new or surprising.

In some cases, the report will be used to confirm (or deny) an expected trend is continuing. The application applies the trend model to the data. If the results don't match expectations, this suggests follow-up action is required. Ideally, it means the model is incorrect, and the trend is changing. The less-than-ideal case is the observation of an unexpected change in the applications providing the source data.

This application decomposes report writing into three distinct steps:

1. **Content:** This is the TOML file with the essential statistical measures.
2. **Structure:** The secondary feature creates an intermediate markup file in Markdown or the RST format. This has an informative structure around the essential content.
3. **Presentation:** The final publication document is created from the structured markup plus any templates or style sheets that are required.

The final presentation is kept separate from the document's content and structure.

An HTML document's final presentation is created by a browser. Using a tool like **Pandoc** to create HTML from Markdown is — properly — replacing one markup language with another markup language.

Creating a PDF file is a bit more complicated. We'll leave this in the extras section at the end of this chapter.

The first step toward creating a nicely formatted document is to create the initial Markdown or ReStructuredText document from the summary.

In many cases, this is easiest done with the **Jinja** package. See

<https://jinja.palletsprojects.com/en/3.1.x/>

One common approach is the following sequence of steps:

1. Write a version of the report using Markdown (or RST).
2. Locate a template and style sheets that produce the desired HTML page when converted by the **Pandoc** or **Docutils** applications.

3. Refactor the source file to replace the content with **Jinja** placeholders. This becomes the template report.
4. Write an application to parse the TOML, then apply the TOML details to the template file.

When using **Jinja** to enable filling in the template, it must be added to the `requirements.txt` file. If **ReStructuredText (RST)** is used, then the **docutils** project is also useful and should be added to the `requirements.txt` file.

If Markdown is used to create the report, then **Pandoc** is one way to handle the conversion from Markdown to HTML. Because **Pandoc** also converts RST to HTML, the **docutils** project is not required.

Because the parsed TOML is a dictionary, fields can be extracted by the Jinja template. We might have a Markdown template file with a structure like the following:

```
# Summary of {{ summary['source']['name'] }}

Created {{ summary['identification']['date'] }}

Some interesting notes about the project...

## X-Variable

Some interesting notes about this variable...

Mean = {{ summary['x']['location']['mean'] }}

etc.
```

The `{{ some-expression }}` constructs are placeholders. This is where Jinja will evaluate the Python expression and replace the placeholders with the resulting value. Because of Jinja's clever implementation, a name like `summary['x']['location']['mean']` can be written as `summary.x.location.mean`, also.

The lines with `#` and `##` are the way Markdown specifies the section headings. For more information on Markdown, see <https://daringfireball.net/projects/markdown/>. Note that there are a large number of Markdown extensions, and it's important to be sure the rendering engine (like Pandoc) supports the extensions you'd like to use.

The Jinja template language has numerous options for conditional and repeating document sections. This includes `{% for name in sequence %}` and `{% if condition %}` constructs to create extremely sophisticated templates. With these constructs, a single template can be used for a number of closely related situations with optional sections to cover special situations.

The application program to inject values from the `summary` object into the template shouldn't be much more complicated than the examples shown on the Jinja basics page. See <https://jinja.palletsprojects.com/en/3.1.x/api/#basics> for some applications that load a template and inject values.

This program's output is a file with a name like `summary_report.md`. This file would be ready for conversion to any of a large number of other formats.

The process of converting a Markdown file to HTML is handled by the **Pandoc** application. See <https://pandoc.org/demos.html>. The command might be as complicated as the following:

```
pandoc -s --toc -c pandoc.css summary_report.md -o summary_report.html
```

The `pandoc.css` file can provide the CSS styles to create a body that's narrow enough to be printed on an ordinary US letter or A4 paper.

The application that creates the `summary_report.md` file can use `subprocess.run()` to execute the **Pandoc** application and create the desired HTML file. This provides a command-line UX that results in a readable document, ready to be distributed.

15.4 Summary

In this chapter we have created a foundation for building and using a statistical model of source data. We've looked at the following topics:

- Designing and building a more complex pipeline of processes for gathering and analyzing data.
- Some of the core concepts behind creating a statistical model of some data.
- Use of the built-in `statistics` library.
- Publishing the results of the statistical measures.

This application tends to be relatively small. The actual computations of the various statistical values leverage the built-in `statistics` library and tend to be very small. It often seems like there's far more programming involved in parsing the CLI argument values, and creating the required output file, than doing the “real work” of this application.

This is a consequence of the way we've been separating the various concerns in data acquisition, cleaning, and analysis. We've partitioned the work into several, isolated stages along a pipeline:

1. Acquiring raw data, generally in text form. This can involve database access or RESTful API access, or complicated file parsing problems.
2. Cleaning and converting the raw data to a more useful, native Python form. This can involve complications of parsing text and rejecting outlier values.
3. Summarizing and analyzing the cleaned data. This can focus on the data model and reporting conclusions about the data.

The idea here is the final application can grow and adapt as our understanding of the data matures. In the next chapter, we'll add features to the summary program to create deeper insights into the available data.

15.5 Extras

Here are some ideas for you to add to this project.

15.5.1 Measures of shape

The measurements of shape often involve two computations for skewness and kurtosis. These functions are not part of Python's built-in `statistics` library.

It's important to note that there are a very large number of distinct, well-understood distributions of data. The normal distribution is one of many different ways data can be distributed.

See

<https://www.itl.nist.gov/div898/handbook/eda/section3/eda366.htm>.

One measure of skewness is the following:

$$g_1 = \frac{\frac{\sum(Y_i - \bar{Y})^3}{N}}{s^3}$$

Where $\bar{Y}$ is the mean, and s is the standard deviation.

A symmetric distribution will have a skewness, g_1 , near zero. Larger numbers indicate a "long tail" opposite a large concentration of data around the mean.

One measure of kurtosis is the following:

$$\text{kurtosis} = \frac{\frac{\sum(Y_i - \bar{Y})^4}{N}}{s^4}$$

The kurtosis for the standard normal distribution is 3. A value larger than 3 suggests more data is in the tails; it's "flatter" or "wider" than the standard normal distribution. A value less than three is "taller" or "narrower" than the standard.

These metrics can be added to the application to compute some additional univariate descriptive statistics.

15.5.2 Creating PDF reports

In the *[Application secondary feature](#)* section we looked at creating a Markdown or RST document with the essential content, some additional information, and an organizational structure. The intent was to use a tool like **Pandoc** to convert the Markdown to HTML. The HTML can be rendered by a browser to present an easy-to-read summary report.

Publishing this document as a PDF requires a tool that can create the necessary output file. There are two common choices:

- Use the **ReportLab** tool: <https://www.reportlab.com/dev/docs/>. This is a commercial product with some open-source components.
- Use the **Pandoc** tool coupled with a LaTeX processing tool.

See *[Preparing a report](#)* of *[Chapter 14, Project 4.2: Creating Reports](#)* for some additional thoughts on using LaTeX to create PDF files. While this involves a large number of separate components, it has the advantage of having the most capabilities.

It's often best to learn the LaTeX tools separately. The TeXLive project maintains a number of tools useful for rendering LaTeX. For macOS users, the MacTeX project offers the required binaries. An online tool like Overleaf is also useful for handling LaTeX. Sort out any problems by creating small `hello_world.tex` example documents to see how the LaTeX tools work.

Once the basics of the LaTeX tools are working, it makes sense to add the **Pandoc** tool to the environment.

Neither of these tools are Python-based and don't use **conda** or **pip** installers.

As noted in *[Chapter 14, Project 4.2: Creating Reports](#)*, there are a lot of components to this tool chain. This is a large number of separate installs that need to be managed. The results, however, can be very nice when a final PDF is created from a few CLI interactions.

15.5.3 Serving the HTML report from the data API

In *[Chapter 12, Project 3.8: Integrated Data Acquisition Web Service](#)* we created a RESTful API service to provide cleaned data.

This service can be expanded to provide several other things. The most notable addition is the HTML summary report.

The process of creating a summary report will look like this:

1. A user makes a request for a summary report for a given time period.
2. The RESTful API creates a “task” to be performed in the background and responds with the status showing the task has been created.
3. The user checks back periodically to see if the processing has finished. Some clever JavaScript programming can display an animation while an application program checks to see if the work is completed.
4. Once the processing is complete, the user can download the final report.

This means two new resources paths will need to be added to the OpenAPI specification. These two new resources are:

- Requests to create a new summary. A POST request creates the task to build a summary and a GET request shows the status. A `2023.03/summarize` path will parallel the `2023.02/creation` path used to create the series.
- Requests for a summary report. A GET request will download a given statistical summary report. Perhaps a `2023.03/report` path would be appropriate.

As we add features to the RESTful API, we need to consider the resource names more and more carefully. The first wave of ideas sometimes fails to reflect the growing understanding of the user’s needs.

In retrospect, the `2023.02/create` path, defined in *[Chapter 12, Project 3.8: Integrated Data Acquisition Web Service](#)*, may not have been the best name.

There's an interesting tension between requests to create a resource and the resulting resource. The request to create a series is clearly distinct from the resulting series. Yet, they can both be meaningfully thought of as instances of "series." The creation request is a kind of *future*: an expectation that will be fulfilled later.

An alternative naming scheme is to use `2023.02/creation` for series, and use `2023.03/create/series` and `2023.03/create/summary` as distinct paths to manage the long-running background that does the work. The task being performed in the background will execute a number of steps:

1. Determine if the request requires new data or existing data. If new data is needed, it is acquired, and cleaned. This is the existing process to acquire the series of data points.
2. Determine if the requested summary does not already exist. (For new data, of course, it will not exist.) If a summary is needed, it is created.

Once the processing is complete, the raw data, cleaned data, and summary can all be available as resources on the API server. The user can request to download any of these resources.

It's essential to be sure each of the components for the task work in isolation before attempting to integrate them as part of a web service. It's far easier to diagnose and debug problems with summary reporting outside the complicated world of web services.